In [1]:
```
import re
import numpy as np
import pandas as pd

df = pd.read_csv("final_movie_psych360.csv")
df_raw = pd.read_csv("Movie_response.csv")
def clean_studio(x):
    if pd.isna(x):
        return np.nan

    x = str(x).strip().lower()
    x = re.sub(r"\s+", " ", x)  # collapse multiple spaces

    if "disney" in x:
        return "Disney"
    elif "pixar" in x:
        return "Pixar"
    elif "dream works" in x or "dreamworks" in x:
        return "DreamWorks"
    else:
        return np.nan
df["guess_studio_clean"] = df["guess_studio"].apply(clean_studio)
print(df["guess_studio_clean"].value_counts(dropna=False))
```

```
guess_studio_clean
Disney        248
DreamWorks    236
Pixar         230
NaN             6
Name: count, dtype: int64
```

In [2]:
```
import numpy as np
import pandas as pd

# 1. LOAD DATA

# Load your responses CSV

# Your column names:
FEATURE_COLS = [
    "feature_human",
    "feature_cgi",
    "feature_tone",
    "feature_setting",
    "feature_action",
    "feature_music"
]

# 2. DEFINE PROTOTYPES
#    (matching EXACT ordering of your feature columns)

PROTOTYPES = {
    "Disney":      np.array([0.30, 0.30, 0.90, 0.40, 0.60, 0.90]),
    "Pixar":       np.array([0.55, 1.00, 0.85, 0.75, 0.47, 0.20]),
    "DreamWorks":  np.array([0.75, 1.00, 0.30, 0.20, 0.82, 0.10])
```

```
}

STUDIOS = ["Disney", "Pixar", "DreamWorks"]


# 3. HELPER FUNCTIONS


def safe_normalize(vec):
    total = np.sum(vec)
    if total == 0:
        return np.ones(len(vec)) / len(vec)
    return vec / total


def shannon_entropy(p):
    p = np.array(p)
    p = np.clip(p, 1e-12, 1)  # avoid log(0)
    return -np.sum(p * np.log(p))


# 4. MODEL COMPUTATION PER ROW

def compute_model_outputs(row):

    # A. MOVIE FEATURE VECTOR (from row itself)
    movie_vec = row[FEATURE_COLS].astype(float).values

    # B. PARTICIPANT PRIORS (handle missing)
    raw_priors = np.array([
        row.get("participant_prior_disney", np.nan),
        row.get("participant_prior_pixar", np.nan),
        row.get("participant_prior_dreamworks", np.nan),
    ], dtype=float)

    raw_priors = np.nan_to_num(raw_priors, nan=0.0)
    priors = safe_normalize(raw_priors)

    # C. LIKELIHOODS USING EXP(-distance^2)
    likelihoods = []
    for studio in STUDIOS:
        prototype = PROTOTYPES[studio]
        d = np.linalg.norm(movie_vec - prototype)**2
        likelihoods.append(np.exp(-d))

    likelihoods = np.array(likelihoods)

    # D. POSTERIOR = PRIORS × LIKELIHOODS
    posterior = safe_normalize(priors * likelihoods)

    # E. MODEL PREDICTION, CONFIDENCE, ENTROPY
    pred_idx = np.argmax(posterior)
    pred_studio = STUDIOS[pred_idx]
    pred_conf = posterior[pred_idx]
    entropy = shannon_entropy(posterior)
```

```
    # F. RETURN AS A SERIES

    return pd.Series({
        "model_posterior_disney": posterior[0],
        "model_posterior_pixar": posterior[1],
        "model_posterior_dreamworks": posterior[2],
        "model_predicted_studio": pred_studio,
        "model_confidence": pred_conf,
        "model_entropy": entropy
    })



# 5. RUN MODEL ACROSS ALL ROWS


model_outputs = df.apply(compute_model_outputs, axis=1)

# Attach to original dataframe
df_final = pd.concat([df, model_outputs], axis=1)

# Save if you want
df_final.to_csv("responses_with_model_posteriors3.csv", index=False)
print("Done! Output saved as responses_with_model_posteriors3.csv.")
```

```
Done! Output saved as responses_with_model_posteriors3.csv.
```

In [14]:

```
#section 1

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df_final = pd.read_csv("responses_with_model_posteriors3.csv")
df_raw = pd.read_csv("Movie_response.csv")


STUDIOS = ["Disney", "Pixar", "DreamWorks"]
df_u = df_final[
    (df_final["participant_knows_movie"] == 0) &
    (df_final["guess_studio_clean"].isin(STUDIOS))
].copy()

studio_to_idx = {s: i for i, s in enumerate(STUDIOS)}

df_u["guess_idx"] = df_u["guess_studio_clean"].map(studio_to_idx)
df_u["model_pred_idx"] = df_u["model_predicted_studio"].map(studio_to_idx)

print(df_u[["guess_idx", "model_pred_idx"]].corr())

confusion = pd.crosstab(
    df_u["guess_studio_clean"],
    df_u["model_predicted_studio"],
    rownames=["Human Guess"],
    colnames=["Model Prediction"]
)
```

```
print(confusion)
```

```
                guess_idx  model_pred_idx
guess_idx        1.000000        0.151709
model_pred_idx   0.151709        1.000000
Model Prediction  Disney  DreamWorks  Pixar
Human Guess
Disney                37           9     32
DreamWorks            24          19     61
Pixar                  8          36     66
```

In [5]:

```
#section 2

# ---- Human modal guess per movie
human_mode = (
    df_u.groupby("movie_name")["guess_studio_clean"]
    .agg(lambda x: x.value_counts().idxmax())
)

# ---- Model modal prediction per movie
model_mode = (
    df_u.groupby("movie_name")["model_predicted_studio"]
    .agg(lambda x: x.value_counts().idxmax())
)

# ---- Ground truth
true_studio = df_u.groupby("movie_name")["studio_true"].first()

# ---- Model entropy per movie
model_entropy = df_u.groupby("movie_name")["model_entropy"].mean()

# ---- Human entropy per movie
def entropy_of_counts(series):
    p = series.value_counts(normalize=True)
    return -np.sum(p * np.log(p + 1e-12))

human_entropy = (
    df_u.groupby("movie_name")["guess_studio_clean"]
    .apply(entropy_of_counts)
)

movie_summary = pd.DataFrame({
    "human_mode": human_mode,
    "model_mode": model_mode,
    "true_studio": true_studio,
    "model_entropy": model_entropy,
    "human_entropy": human_entropy
})
```

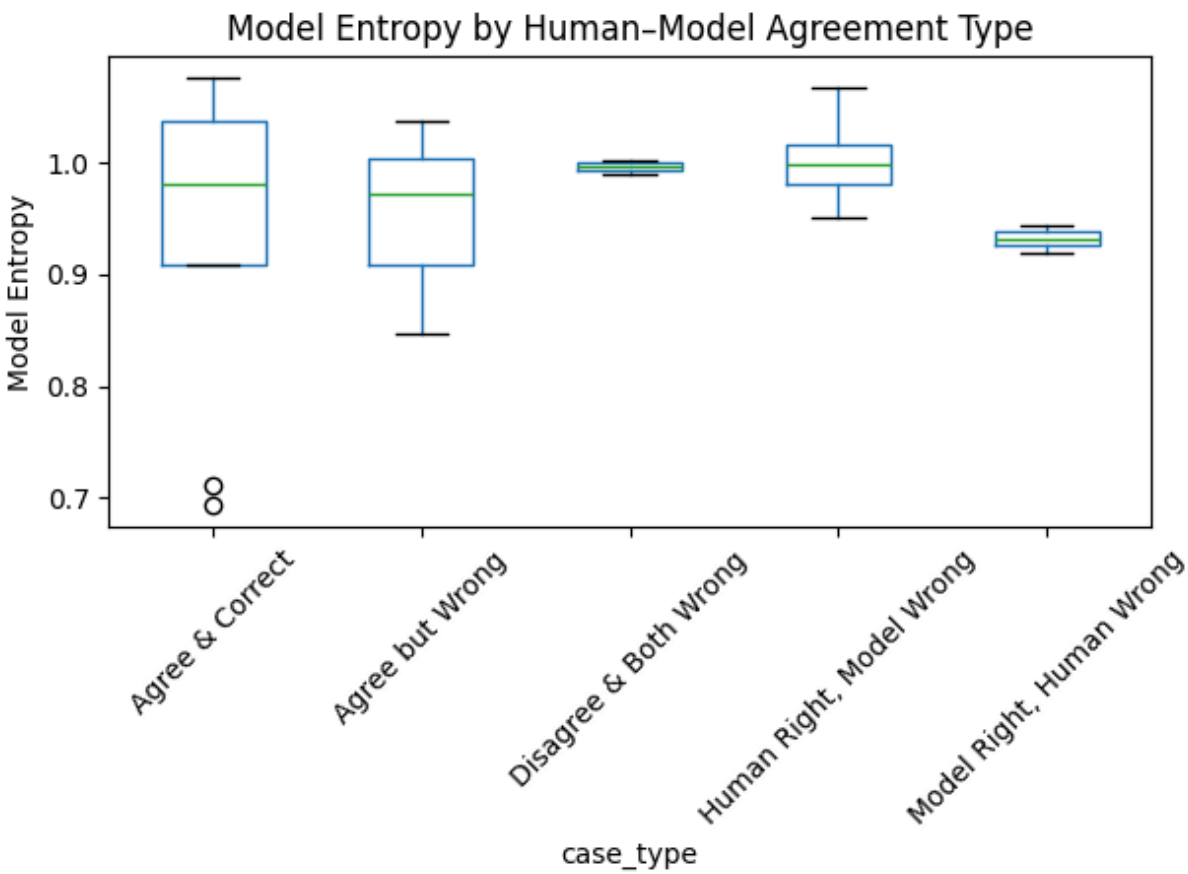
In [6]:

```
movie_summary["human_correct"] = movie_summary["human_mode"] == movie_summar
movie_summary["model_correct"] = movie_summary["model_mode"] == movie_summar
movie_summary["human_model_agree"] = movie_summary["human_mode"] == movie_su

def classify_case(row):
    if row["human_model_agree"] and row["human_correct"]:
```

```
        return "Agree & Correct"
    elif row["human_model_agree"] and not row["human_correct"]:
        return "Agree but Wrong"
    elif not row["human_model_agree"] and row["human_correct"]:
        return "Human Right, Model Wrong"
    elif not row["human_model_agree"] and row["model_correct"]:
        return "Model Right, Human Wrong"
    else:
        return "Disagree & Both Wrong"

movie_summary["case_type"] = movie_summary.apply(classify_case, axis=1)
```

In [7]:
```
plt.figure(figsize=(8, 5))
movie_summary.boxplot(column="model_entropy", by="case_type", grid=False)
plt.title("Model Entropy by Human–Model Agreement Type")
plt.suptitle("")
plt.ylabel("Model Entropy")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
<Figure size 800x500 with 0 Axes>
```

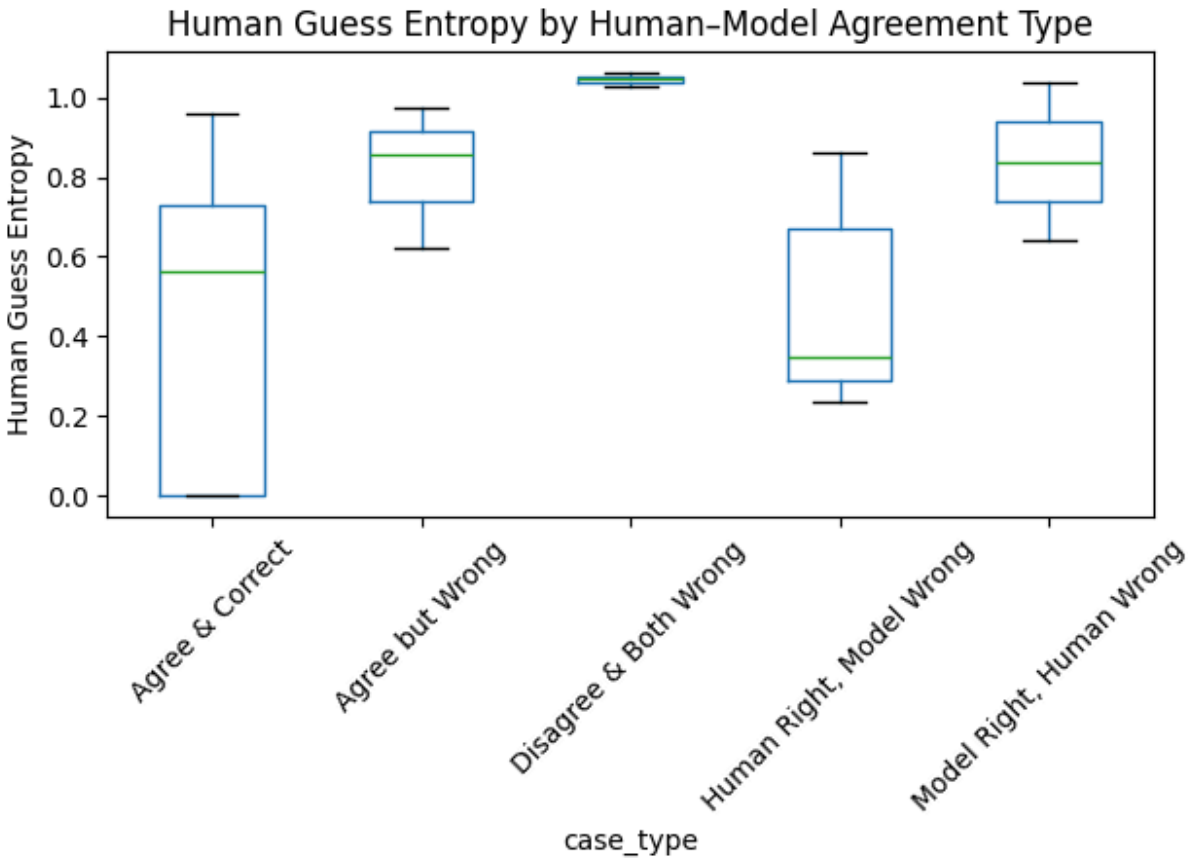


In [ ]:

In [8]:
```
plt.figure(figsize=(8, 5))
movie_summary.boxplot(column="human_entropy", by="case_type", grid=False)
plt.title("Human Guess Entropy by Human–Model Agreement Type")
plt.suptitle("")
```

```
plt.ylabel("Human Guess Entropy")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
<Figure size 800x500 with 0 Axes>
```



In [23]:

```
from scipy.stats import ttest_ind

# Split movies by agreement type
agree_correct = movie_summary[
    movie_summary["case_type"] == "Agree & Correct"
]["model_entropy"]

disagree = movie_summary[
    movie_summary["case_type"] != "Agree & Correct"
]["model_entropy"]

# Independent samples t-test
t_stat, p_val = ttest_ind(
    agree_correct,
    disagree,
    equal_var=False,   # Welch's t-test (safer)
    nan_policy="omit"
)

print(f"T-test comparing model entropy:")
print(f"t = {t_stat:.2f}, p = {p_val:.4f}")
print(f"Mean entropy (Agree & Correct): {agree_correct.mean():.3f}")
print(f"Mean entropy (Disagreement): {disagree.mean():.3f}")
```

```
T-test comparing model entropy:
t = -0.91, p = 0.3860
Mean entropy (Agree & Correct): 0.937
Mean entropy (Disagreement): 0.983
```

In [17]:

```python
#section 3
import pandas as pd
import numpy as np
import re

# 1. LOAD FILES
adf_raw = pd.read_csv("Movie_response.csv")
df_final = pd.read_csv("responses_with_model_posteriors3.csv")

# 2. MOVIE ORDER (FIXED SURVEY DESIGN)
amovie_order = [
    "Atlantis",
    "Beauty and the Beast",
    "Bolt",
    "Big Hero 6",
    "Brave",
    "Coco",
    "Finding Nemo",
    "How to Train Your Dragon",
    "Inside Out",
    "Kung Fu Panda",
    "Madagascar",
    "Megamind",
    "Moana",
    "Mulan",
    "Onward",
    "Rise of the Guardians",
    "Shrek",
    "The Incredibles",
    "The Little Mermaid",
    "The Prince of Egypt",
    "The Road to El Dorado",
    "Toy Story",
    "Up",
    "Wreck it Ralph"
]

# 3. IDENTIFY MOVIE QUESTION COLUMNS
aquestion_cols = [
    c for c in df_raw.columns
    if "Which Animation Studio do you think this movie belongs to?" in c
]

assert len(question_cols) == len(movie_order), "Movie count mismatch!"

# 4. MELT GOOGLE FORM TO LONG FORMAT
adf_conf = df_raw.melt(
    id_vars=["Timestamp"],
    value_vars=question_cols,
    var_name="question",
    value_name="response"
```

```
)

# 5. EXTRACT CONFIDENCE (1–7)
adef extract_confidence_1to7(cell):
    if pd.isna(cell):
        return np.nan
    match = re.search(r"\b([1-7])\b", str(cell))
    return int(match.group(1)) if match else np.nan

df_conf["confidence"] = df_conf["response"].apply(extract_confidence_1to7)

# 6. MAP QUESTIONS → MOVIES
aquestion_to_movie = dict(zip(question_cols, movie_order))
df_conf["movie_name"] = df_conf["question"].map(question_to_movie)

assert df_conf["movie_name"].isna().sum() == 0, "Movie mapping failed"

# 7. CREATE TRIAL INDEX (THIS IS THE KEY FIX)
adf_conf["trial_index"] = df_conf.groupby("Timestamp").cumcount()
df_final["trial_index"] = df_final.groupby("participant_id").cumcount()

# 8. MERGE CONFIDENCE (ONLY MERGE — DO NOT ADD OTHERS)
adf_final = df_final.merge(
    df_conf[["trial_index", "movie_name", "confidence"]],
    on=["trial_index", "movie_name"],
    how="left"
)

# 9. SANITY CHECKS
aprint(df_final["confidence"].describe())
print("Missing confidence proportion:", df_final["confidence"].isna().mean()

# 10. OPTIONAL: CONFIDENCE ↔ MODEL ENTROPY (UNFAMILIAR ONLY)
adf_u = df_final[df_final["participant_knows_movie"] == 0]
print(df_u[["confidence", "model_entropy"]].corr())
```

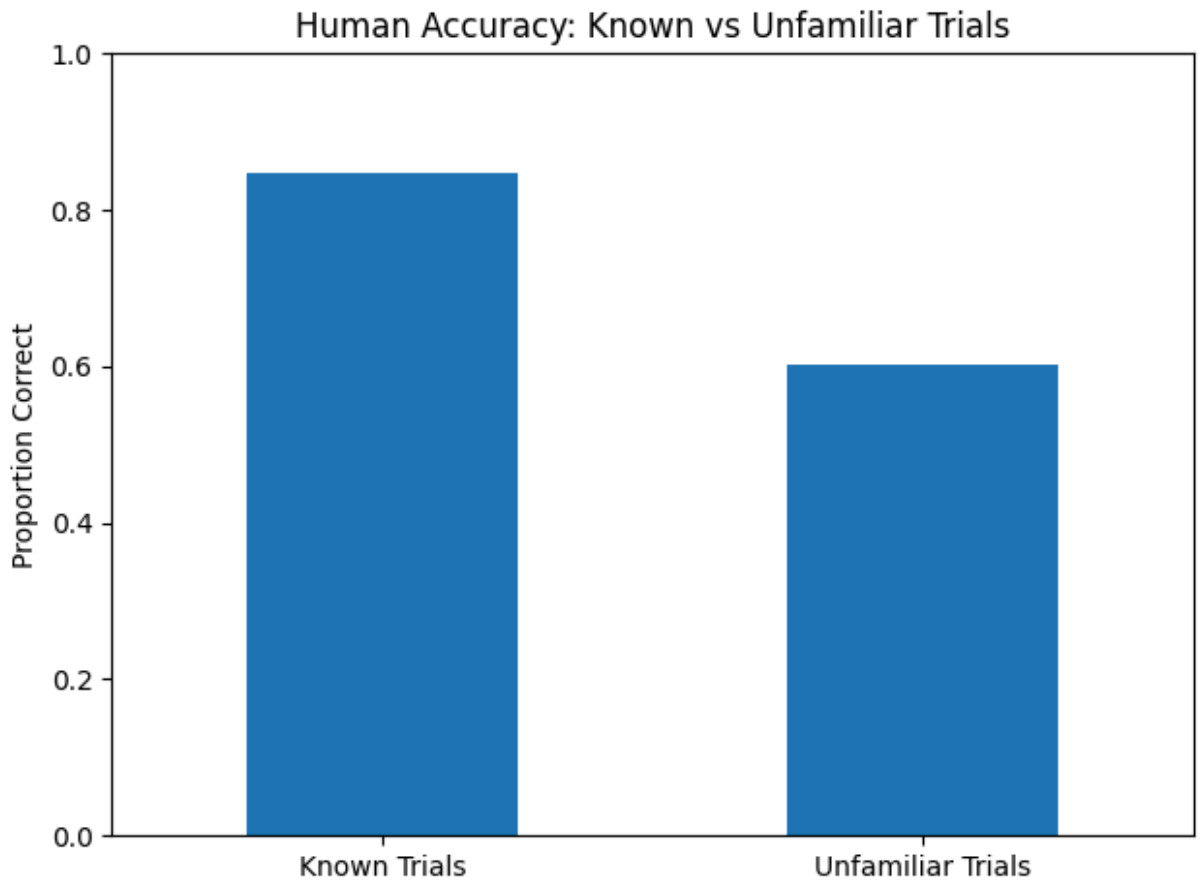
```
count    11070.000000
mean         5.392954
std          1.726661
min          1.000000
25%          4.000000
50%          6.000000
75%          7.000000
max          7.000000
Name: confidence, dtype: float64
Missing confidence proportion: 0.4875
               confidence  model_entropy
confidence       1.000000       0.005434
model_entropy    0.005434       1.000000
```

In [18]:

```
#section 4
df_final["human_correct"] = (
    df_final["guess_studio_clean"] == df_final["studio_true"]
)
df_known = df_final[df_final["participant_knows_movie"] == 1]
df_unknown = df_final[df_final["participant_knows_movie"] == 0]
```

```
accuracy_summary = pd.DataFrame({
    "Accuracy": [
        df_known["human_correct"].mean(),
        df_unknown["human_correct"].mean()
    ]
}, index=["Known Trials", "Unfamiliar Trials"])

print("\nHuman accuracy by trial type:")
print(accuracy_summary)

accuracy_summary.plot(kind="bar", legend=False)
plt.ylabel("Proportion Correct")
plt.title("Human Accuracy: Known vs Unfamiliar Trials")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

studio_to_idx = {"Disney": 0, "Pixar": 1, "DreamWorks": 2}

df_known = df_known[df_known["guess_studio_clean"].isin(studio_to_idx)]
df_known["guess_idx"] = df_known["guess_studio_clean"].map(studio_to_idx)

df_known["model_posterior_chosen"] = df_known.apply(
    lambda r: r[
        ["model_posterior_disney",
         "model_posterior_pixar",
         "model_posterior_dreamworks"]
    ].values[int(r["guess_idx"])],
    axis=1
)

print("\nKnown trials: Posterior (chosen studio) ↔ Accuracy")
print(df_known[["model_posterior_chosen", "human_correct"]].corr())
```

```
Human accuracy by trial type:
                   Accuracy
Known Trials       0.847291
Unfamiliar Trials  0.602740
```

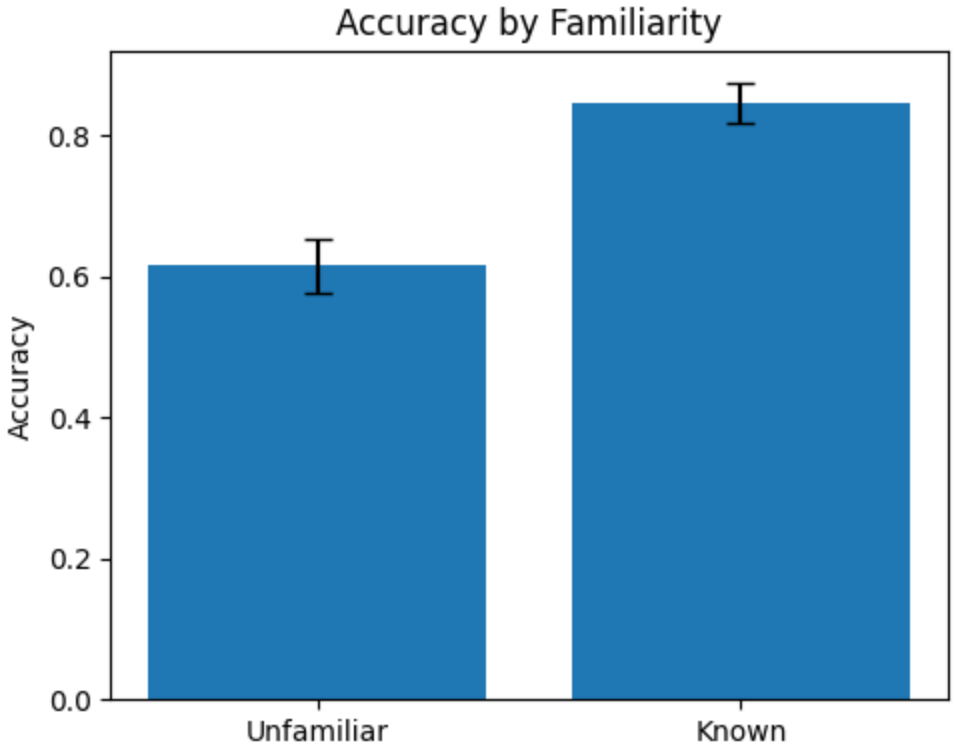



```
Known trials: Posterior (chosen studio) ↔ Accuracy
                        model_posterior_chosen  human_correct
model_posterior_chosen                1.000000       0.203672
human_correct                         0.203672       1.000000
```

In [22]:

```
import numpy as np
import matplotlib.pyplot as plt

# compute accuracy per participant
acc_by_participant = (
    df_final
    .assign(correct=df_final["guess_studio_clean"] == df_final["studio_true"
    .groupby(["participant_id", "participant_knows_movie"])["correct"]
    .mean()
    .reset_index()
)

# compute means and SEMs
summary = (
    acc_by_participant
    .groupby("participant_knows_movie")["correct"]
    .agg(["mean", "sem"])
)

# plot
plt.figure(figsize=(5, 4))
plt.bar(
    ["Unfamiliar", "Known"],
    summary["mean"],
```

```
    yerr=summary["sem"],
    capsize=5
)
plt.ylabel("Accuracy")
plt.title("Accuracy by Familiarity")
plt.tight_layout()
plt.show()
```



In [19]:

```
#section 5

import pandas as pd
import numpy as np

# Studio prototypes (must match your model exactly)
PROTOTYPES = {
    "Disney":      np.array([0.30, 0.30, 0.90, 0.40, 0.60, 0.90]),
    "Pixar":       np.array([0.55, 1.00, 0.85, 0.75, 0.47, 0.20]),
    "DreamWorks":  np.array([0.75, 1.00, 0.30, 0.20, 0.82, 0.10])
}

studios = list(PROTOTYPES.keys())

# Pairwise distances
distance_matrix = pd.DataFrame(index=studios, columns=studios, dtype=float)

for s1 in studios:
    for s2 in studios:
        distance_matrix.loc[s1, s2] = np.linalg.norm(
            PROTOTYPES[s1] - PROTOTYPES[s2]
        )

print("Prototype distance matrix (lower = more overlap):")
```

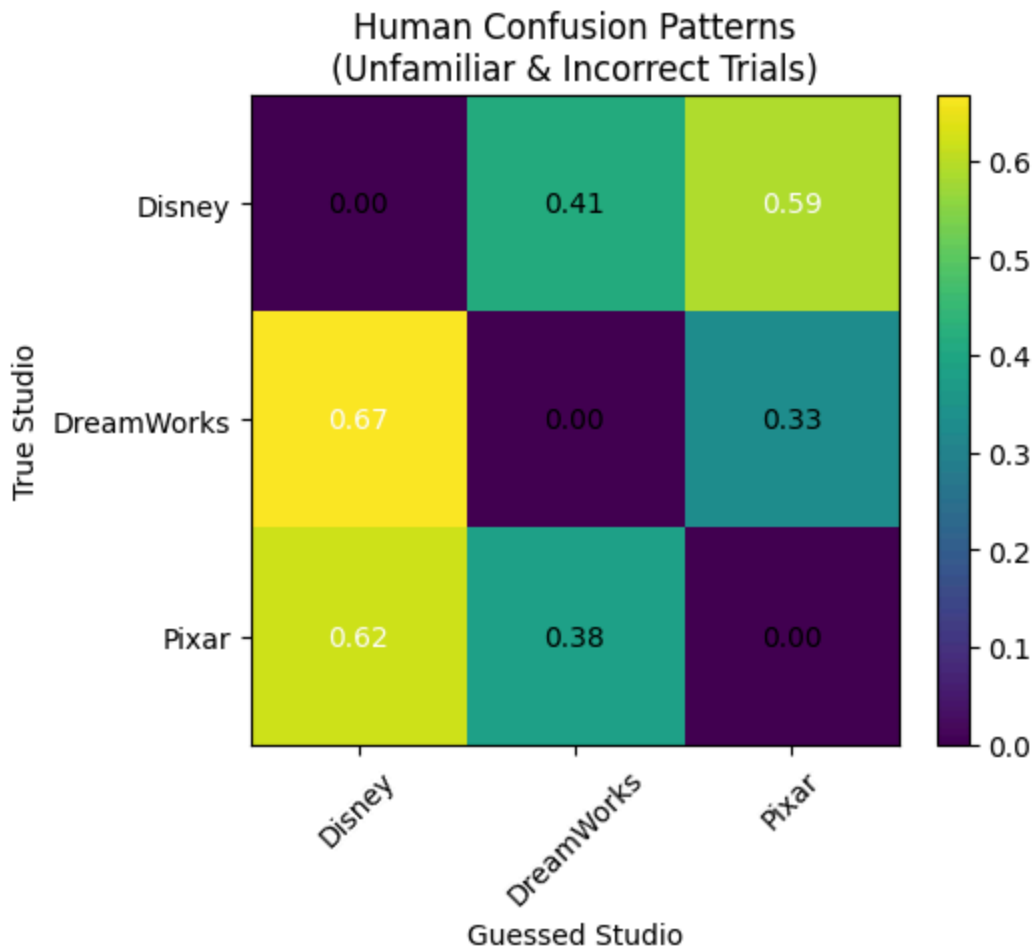
```
print(distance_matrix)



df_u = df_final[
    (df_final["participant_knows_movie"] == 0) &
    (df_final["guess_studio_clean"].isin(["Disney", "Pixar", "DreamWorks"]))
].copy()

# Only wrong trials
df_wrong = df_u[df_u["guess_studio_clean"] != df_u["studio_true"]]

confusion_matrix = pd.crosstab(
    df_wrong["studio_true"],
    df_wrong["guess_studio_clean"],
    rownames=["True Studio"],
    colnames=["Guessed Studio"]
)

print("\nHuman confusion matrix (unfamiliar & incorrect trials only):")
print(confusion_matrix)



confusion_norm = confusion_matrix.div(
    confusion_matrix.sum(axis=1), axis=0
)

print("\nNormalized confusion probabilities:")
print(confusion_norm)
```

```
Prototype distance matrix (lower = more overlap):
              Disney     Pixar  DreamWorks
Disney      0.000000  1.088301    1.334504
Pixar       1.088301  0.000000    0.881760
DreamWorks  1.334504  0.881760    0.000000

Human confusion matrix (unfamiliar & incorrect trials only):
Guessed Studio  Disney  DreamWorks  Pixar
True Studio
Disney               0         570    810
DreamWorks         720           0    360
Pixar              630         390      0

Normalized confusion probabilities:
Guessed Studio    Disney  DreamWorks     Pixar
True Studio
Disney          0.000000    0.413043  0.586957
DreamWorks      0.666667    0.000000  0.333333
Pixar           0.617647    0.382353  0.000000
```

In [24]:

```
import matplotlib.pyplot as plt

a# HEATMAP: NORMALIZED HUMAN CONFUSION MATRIX
a
plt.figure(figsize=(6, 5))
```

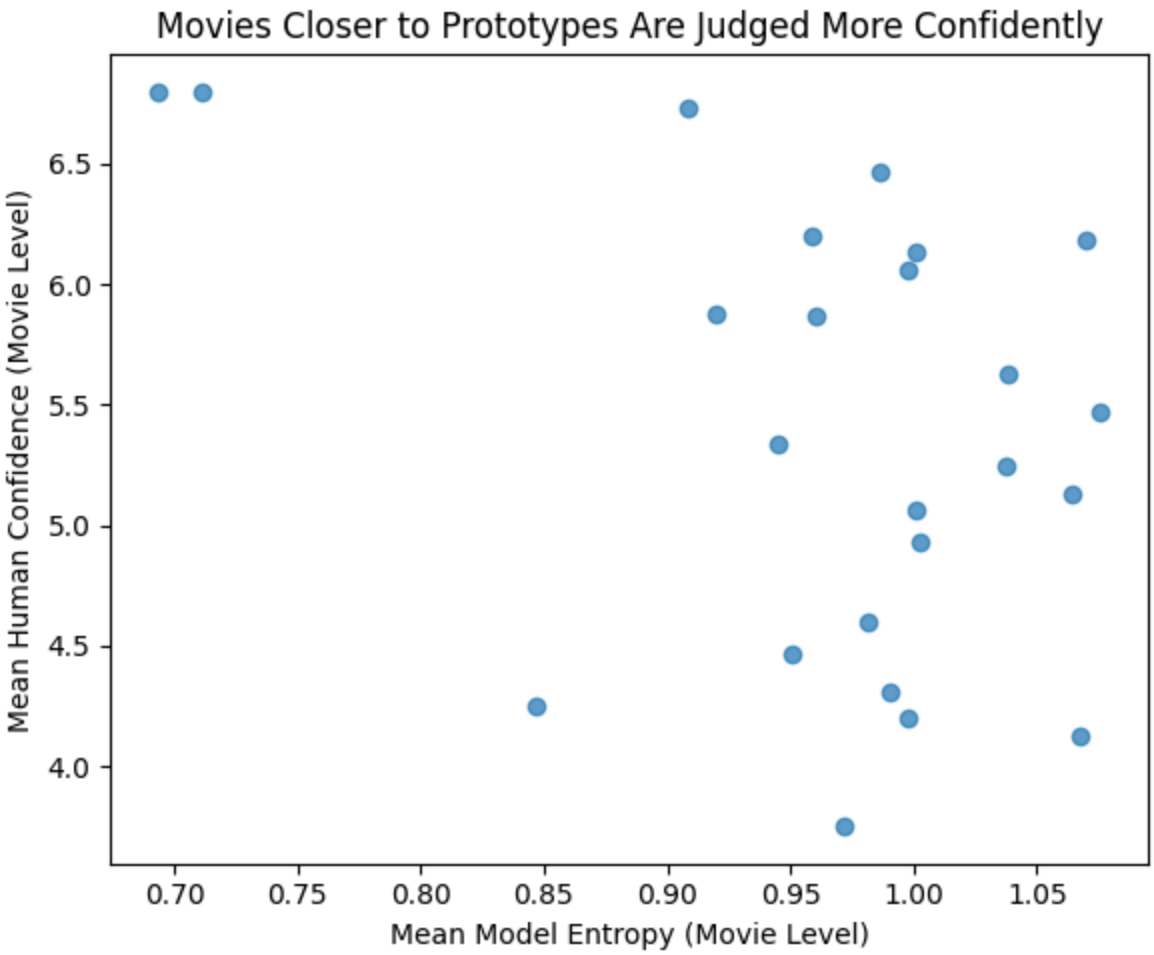
```python
# Plot heatmap
im = plt.imshow(confusion_norm.values, cmap="viridis")

# Add colorbar
plt.colorbar(im, fraction=0.046, pad=0.04)

# Axis labels and ticks
plt.xticks(
    ticks=np.arange(len(confusion_norm.columns)),
    labels=confusion_norm.columns,
    rotation=45
)
plt.yticks(
    ticks=np.arange(len(confusion_norm.index)),
    labels=confusion_norm.index
)

plt.xlabel("Guessed Studio")
plt.ylabel("True Studio")
plt.title("Human Confusion Patterns\n(Unfamiliar & Incorrect Trials)")

# Annotate each cell with probability
for i in range(confusion_norm.shape[0]):
    for j in range(confusion_norm.shape[1]):
        value = confusion_norm.values[i, j]
        if not np.isnan(value):
            plt.text(
                j, i, f"{value:.2f}",
                ha="center", va="center",
                color="white" if value > 0.5 else "black"
            )

plt.tight_layout()
plt.show()
```



In [21]:
```
#section 6

df_u = df_final[
    (df_final["participant_knows_movie"] == 0) &
    (~df_final["confidence"].isna())
].copy()

movie_confidence_entropy = (
    df_u
    .groupby("movie_name")
    .agg({
        "model_entropy": "mean",
        "confidence": "mean"
    })
    .rename(columns={
        "model_entropy": "mean_model_entropy",
        "confidence": "mean_human_confidence"
    })
)
print("Movie-level correlation: Model Entropy ↔ Human Confidence")
print(movie_confidence_entropy.corr())
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 5))
plt.scatter(
    movie_confidence_entropy["mean_model_entropy"],
```

```
    movie_confidence_entropy["mean_human_confidence"],
    alpha=0.7
)
plt.xlabel("Mean Model Entropy (Movie Level)")
plt.ylabel("Mean Human Confidence (Movie Level)")
plt.title("Movies Closer to Prototypes Are Judged More Confidently")
plt.tight_layout()
plt.show()
```

```
Movie-level correlation: Model Entropy ↔ Human Confidence
                      mean_model_entropy  mean_human_confidence
mean_model_entropy                1.00000              -0.38312
mean_human_confidence            -0.38312               1.00000
```



In [ ]: